

第一次课程学习总结 ——计算机基础快速入门与工具实践

本报告围绕研究生阶段学习规划、Linux、Markdown、 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 以及 Python/C++ 展开，既总结了课堂内容，也可作为直接使用的个人速查手册。

课程名称： 学业基本情况介绍与计算机基本使用

姓名： 任鹏程

学号： 2026201673

专业： 计算机科学与技术

提交日期： 2026 年 7 月 18 日

摘要

第一次课程从研究生阶段的培养目标出发，介绍了课程学习、科研评价、课题组协作和计算机工具使用等内容。本文首先梳理课堂要点并形成个人行动计划；随后围绕 Linux、Markdown、 $\text{\LaTeX}$ 及编程语言进行系统归纳。在语言部分，除复习 Python 与 C++ 外，还总结了我较为熟练的 Java、JavaScript、SQL 与 Web 开发技术。报告不把工具视为彼此孤立的软件，而是将它们组织为“终端完成任务、Markdown 记录过程、 $\text{\LaTeX}$ 沉淀正式成果、程序语言解决计算问题”的完整 workflow。通过本次整理，我对研究生阶段如何管理学习、科研、资料和计算环境有了更清晰的认识。

关键词：研究生学习；Linux；Markdown； $\text{\LaTeX}$ ；Python；C++

目录

一、 第一次课程内容总结与个人认识	1	四、 $\text{\LaTeX}$ 学习总结与本文实践	7
1.1 研究生阶段的核心任务	1	4.1 基本认识	7
1.2 科研进程与阶段性规划	1	4.2 最小中文文档与编译方式	7
1.3 课题组中的协作与沟通	2	4.3 文档结构、环境和命令	8
1.4 计算机管理与正确使用	2	4.4 本文使用的排版功能	8
二、 Linux 操作系统与个人速查手册	3	五、 个人编程语言基础与复习总结	9
2.1 Linux 的基本认识	3	5.1 已有语言基础概况	9
2.2 目录结构与路径	3	5.2 Java 开发经验总结	10
2.3 常用命令速查	4	5.3 JavaScript 与 Web 开发经验总结	11
2.4 权限、软件安装与进程	4	5.4 Python 与 C++ 的定位	11
2.5 一个可复用的科研工作流	5	5.5 Python 知识框架	11
三、 Markdown 学习总结	5	5.6 C++ 知识框架	12
3.1 Markdown、CommonMark 与 GFM	5	5.7 面向科研的联合 workflow	13
3.2 核心语法示例	5	六、 总结与后续计划	13
3.3 扩展功能：数学公式与图表	6	参考资料	15
3.4 Markdown 的适用边界	7		

一、第一次课程内容总结与个人认识

1.1 研究生阶段的核心任务

课堂首先比较了本科与研究生阶段的差异。本科阶段的学习往往以课程、考试和统一安排为主，而研究生阶段更强调自主学习、问题意识和科研产出。课程数量可能减少，但对阅读、实验、写作和汇报的要求明显提高。因此，“课少”并不意味着任务少，而是意味着需要自行安排的时间更多。

课堂中最重要的提醒是：**科研是研究生阶段的核心任务**。课程成绩需要达到基本要求，但真正体现研究能力的，是能否发现问题、阅读文献、设计方案、完成实验并把结果表达为论文或报告。科研成果通常也是奖学金评定、毕业审核和后续深造的重要依据。

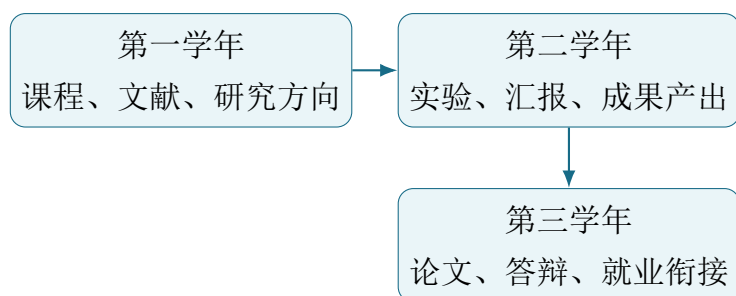
表 1: 本科阶段与研究生阶段学习方式对比

维度	本科阶段	研究生阶段
学习驱动	课程表、考试与统一作业	研究问题、项目任务与个人规划
知识获取	教材和课堂讲授为主	论文、文档、实验和讨论并重
评价方式	课程成绩占比较高	科研成果、汇报、论文与综合表现
时间管理	安排相对固定	自主时间多，更考验计划与执行
最终目标	掌握专业基础知识	形成独立分析和解决问题的能力

1.2 科研进程与阶段性规划

结合课堂内容，我把这三年的硕士学习概括为以下三步：

- 第一学年：打基础、定方向。**完成课程学习，熟悉课题组研究方向，阅读高质量论文，掌握必要的实验工具，并在导师指导下确定选题。
- 第二学年：做研究、出成果。**围绕选题开展实验，及时记录数据与失败原因，形成阶段性报告，争取完成论文、专利或学术汇报。
- 第三学年：收尾与衔接。**完成学位论文、送审和答辩，同时结合个人目标准备求职、考公考编或继续深造。



上述规划不是机械的时间表。例如，文献阅读与写作应贯穿整个培养过程，而不能等到毕业前才开始。对我而言，第一学期尤其要做好研究方向调研和工具准备，尽快建立可重复的工作环境。

1.3 课题组中的协作与沟通

课堂强调了团结、责任感和规则意识。课题组不仅是完成个人论文的场所，也是一个协作共同体。按时到实验室、维护公共资源、主动汇报进度、帮助同学解决问题，都会直接影响团队效率。

与导师沟通时，应尽量做到“提前联系、带着问题、说明尝试、记录结论”。有课程时不临时打扰；其他时间需要联系时，应尽量提前预约。遇到科研问题，可以先查阅文献、文档和既往记录，再把“现象—已做尝试—当前判断—希望获得的帮助”整理清楚。小事先与学长学姐沟通，需要导师决策的事项再正式汇报。

1.4 计算机管理与正确使用

课堂第二部分介绍了计算机的日常管理。我的理解是，研究生电脑不仅是娱乐和办公设备，更是科研数据与实验环境的载体，其稳定性、可恢复性和可复现性都很重要。

- **保持系统可靠：**使用正版或来源可靠的软件，及时更新系统和安全补丁，不随意安装“管家类”捆绑软件。
- **规范卸载与清理：**使用系统卸载功能或可靠卸载工具；用 WizTree 等工具分析空间占用，而不是随意删除未知系统文件。
- **选择合适工具：**文字排版使用 Office 或 L^AT_EX，表格和数据处理使用 Excel 或专业工具，尽量让工具与任务匹配。
- **提高操作效率：**熟悉 PowerToys、Windows Terminal 等以及常用快捷键；对重复工作尽量脚本化。
- **保护科研资料：**重要论文、代码和实验数据至少保留两份副本；代码使用 Git 记录版本，数据按日期和项目分类。

培养建立统一的文件目录与命名规则的习惯；为代码项目使用 Git；定期整理实验记录；每月检查一次备份；把常用命令和排错经验持续补充到个人 Markdown 知识库中。

二、Linux 操作系统与个人速查手册

2.1 Linux 的基本认识

操作系统负责管理处理器、内存、存储设备、文件和外设，并向应用程序提供统一接口。课堂中学习到常见操作系统大致分为 Windows NT 系列和类 Unix 系统。Linux 是一种类 Unix 操作系统内核，日常所说的 Ubuntu、Debian、Fedora、Arch Linux 等，实际是由 Linux 内核、GNU 工具、软件包管理器和桌面环境等共同组成的发行版。

Linux 的主要特点包括：多用户、多任务；权限边界清晰；文本工具丰富；适合自动化；服务器生态成熟。科研中常见的远程服务器、高性能计算平台、容器和深度学习环境大多与 Linux 密切相关。对初学者而言，Ubuntu 或 Debian 资料丰富、软件管理成熟，适合作为入门选择。这也是后续我们进行科研主要的基础平台。

2.2 目录结构与路径

Linux使用一棵以 “/” 为根的目录树。理解目录用途，有助于避免误操作。

表 2: 常见 Linux 目录

目录	用途	目录	用途
/home	普通用户主目录	/etc	系统与服务配置
/usr	用户程序、库和共享资源	/var	日志、缓存等可变数据
/tmp	临时文件	/dev	设备文件
/proc	内核和进程的虚拟信息	/mnt	临时挂载点

绝对路径从根目录开始，例如 `/home/user/project`；相对路径以当前目录为起点。`.` 表示当前目录，`..` 表示上一级目录，`~` 表示当前用户主目录。

2.3 常用命令速查

表 3: 个人常用命令速查表

任务	命令示例	说明
查看位置	<code>pwd</code>	显示当前工作目录
列出文件	<code>ls -lah</code>	包括隐藏项，并显示易读大小
切换目录	<code>cd /path/to/dir</code>	进入指定目录
创建目录	<code>mkdir -p data/raw</code>	递归创建多级目录
复制文件	<code>cp -r src backup/</code>	递归复制目录
移动/改名	<code>mv old.txt new.txt</code>	移动或重命名
查看文本	<code>less file.log</code>	分页查看大文件
搜索文件	<code>find . -name '*.py'</code>	按名称查找文件
搜索内容	<code>grep -R 'error' logs/</code>	递归查找文本
磁盘空间	<code>df -h; du -sh *</code>	查看分区和目录占用
进程信息	<code>ps aux; top</code>	查看进程与资源使用
远程登录	<code>ssh user@host</code>	通过 SSH 连接服务器
传输文件	<code>scp file user@host:/path</code>	在本地与服务器间复制
查看帮助	<code>man command</code>	阅读命令手册

管道符 `|` 可以把前一个命令的输出交给后一个命令处理。例如：

demo 1: 查找最占空间的五个子目录

```
1 du -sh ./ * | sort -hr | head -n 5
```

重定向符 `>` 覆盖写入文件，`>>` 追加写入，`2>` 重定向错误输出。组合命令之前应先在测试目录验证，尤其要谨慎使用带有递归参数的删除命令。

2.4 权限、软件安装与进程

执行 `ls -l` 时，可以看到类似 `-rwxr-xr--` 的权限字符串。前三位对应所有者，中间三位对应所属组，最后三位对应其他用户；`r`、`w`、`x` 分别表示读、写、执行。

demo 2: 权限与软件包管理示例

```
1 # 为脚本增加所有者执行权限
2 chmod u+x run.sh
3
4 # Ubuntu/Debian 更新索引并安装 Git
5 sudo apt update
6 sudo apt install git
7
```

```
8 # 查询占用 8000 端口的进程
9 ss -lntp | grep ':8000'
```

`sudo` 会以更高权限执行命令，应只在明确知道命令影响范围时使用。软件安装优先使用发行版的软件包管理器，避免从不明网站下载并直接执行脚本。

2.5 一个可复用的科研 workflow

下面给出我准备采用的 Linux 项目目录与操作流程：

demo 3: 建立规范的项目目录

```
1 mkdir -p experiment/{data/raw,data/processed,src,results,docs}
2 cd experiment
3 git init
4 printf "data/raw/\nresults/tmp/\n" > .gitignore
5 git add .gitignore
6 git commit -m "init project structure"
```

原始数据只读保存，清洗后的数据放入 `data/processed`，程序放入 `src`，可复现实验结果放入 `results`，说明文档和报告放入 `docs`。这种结构能降低文件混乱和误覆盖的风险。

三、 Markdown 学习总结

3.1 Markdown、CommonMark 与 GFM

Markdown 是一种轻量级标记语言，目标是让纯文本既易读，又能转换为 HTML 等格式。由于早期 Markdown 的边界行为没有完全统一，不同解析器可能产生不同结果。CommonMark 通过规范化语法与测试用例，减少了解析差异；GitHub Flavored Markdown (GFM) 建立在 CommonMark 基础上，增加了表格、任务列表、删除线和自动链接等 GitHub 常用功能。

3.2 核心语法示例

demo 4: Markdown 常用语法

```
1 # 一级标题
2 ## 二级标题
3
4 这是**粗体**、*斜体*和`行内代码`。
5
```

表 4: CommonMark 与 GFM 的关系

比较项	CommonMark	GFM
定位	通用、明确的 Markdown 规范	面向 GitHub 场景的扩展规范
核心语法	标题、段落、列表、引用、链接、代码等	继承 CommonMark 核心语法
典型扩展	不统一要求扩展语法	表格、任务列表、删除线、自动链接
适用场景	跨平台文档与解析器实现	README、Issue、Pull Request、项目协作

```
6 - 无序列表
7 1. 有序列表
8 > 引用内容
9
10 [链接文字](https://example.com)
11 ![图片说明](image.png)
12
13 | 工具 | 用途 |
14 | --- | --- |
15 | Git | 版本控制 |
16
17 - [x] 已完成
18 - [ ] 待完成
```

写 Markdown 时，标题符号后应留空格；不同内容块之间保留空行；代码围栏应标明语言。这样既提高源文件可读性，也能获得稳定的渲染结果。

3.3 扩展功能：数学公式与图表

部分 Markdown 渲染器支持 L^AT_EX 数学语法。例如行内公式写作 $E=mc^2$ ，独立公式可以写成：

demo 5: Markdown 中的数学公式

```
1 $$
2 \sum_{n=1}^{\infty}\frac{1}{n^2}=\frac{\pi^2}{6}
3 $$
```


其渲染结果为

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}.$$

Mermaid适合流程图、时序图和状态图；PlantUML 更偏向软件工程中的 UML 图。它们通常通过代码描述图形，便于版本控制和后期修改。例如：

demo 6: Mermaid 流程图源码

```
1  ``mermaid
2  flowchart LR
3      A[提出问题] --> B[查阅文献]
4      B --> C[设计实验]
5      C --> D[分析结果]
6  ``
```

需要注意的是，公式、Mermaid、PlantUML、嵌入 HTML 都属于扩展功能，不同平台的支持程度并不一致。为了提高可移植性，提交前必须在目标平台预览；面向长期保存的核心内容，应尽量保留纯文本含义。

3.4 Markdown 的适用边界

Markdown特别适合 README、实验日志、会议记录、个人知识库和简单网页内容。它的优势是输入快、版本差异清晰、无需复杂排版操作；不足是精细分页、浮动体、交叉引用、复杂公式编号和正式出版排版能力有限。因此，后续科研中我会用 Markdown 记录过程，用 L^AT_EX 整理正式报告和论文。

四、 L^AT_EX 学习总结与本文实践

4.1 基本认识

T_EX 是高质量排版系统，L^AT_EX 在其基础上提供了文档类、宏命令和大量扩展包。它可以理解为一种面向文档结构的“标记语言 + 排版引擎 workflow”：作者描述标题、章节、公式、表格和引用的逻辑结构，由编译引擎计算最终版面。

与“看到什么就直接调整什么”的文字处理软件相比，L^AT_EX 更强调内容与格式分离。其优点是数学公式质量高、编号和交叉引用自动化、长文档样式统一、源文件适合 Git 管理；缺点是初期需要学习语法，错误信息不够直观，复杂表格或自由版式的调整成本较高。

4.2 最小中文文档与编译方式

demo 7: 可编译的最小中文文档（我的第一份 L^AT_EX demo）

```

1 \documentclass[UTF8]{ctexart}
2
3 \title{我的第一份 \LaTeX{} 文档}
4 \author{任鹏程}
5 \date{\today}
6
7 \begin{document}
8 \maketitle
9 你好，世界！
10 \end{document}

```

中文文档建议使用 XeLaTeX 编译。若使用 VS Code，可以安装 LaTeX Workshop 扩展，并让构建配方调用 XeLaTeX；也可以在命令行执行：

demo 8: 编译命令

```

1 xelatex report.tex
2 xelatex report.tex # 第二次用于更新目录、引用和页码

```

ctexart 本身已经提供中文支持，所以使用 `\documentclass{ctexart}` 后，不需要再写 `\usepackage{ctex}`。二者重复使用可能触发“Package ctex can not be loaded with ctexart”一类错误。

4.3 文档结构、环境和命令

L^AT_EX 源文件由导言区和正文区构成。`\documentclass` 选择文档类型；`\usepackage` 加载功能；`\begin{document}` 与 `\end{document}` 包围正文。章节使用 `\section` 等命令，列表、公式、表格、图片使用相应环境。

例如，带编号的公式可以写为

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right], \quad (1)$$

正文使用“式 1”引用后，即使中途增加其他公式，编号也会自动更新。

4.4 本文使用的排版功能

本报告本身就是一次 L^AT_EX 实践，主要使用了以下功能：

- 使用 ctexart 处理中文断行、字号和章节标题；

表 5: 常用 L^AT_EX 功能

功能	写法	说明
标题信息	<code>\title</code> 、 <code>\author</code>	需用 <code>\maketitle</code> 输出
章节	<code>\section {标题}</code>	自动编号并进入目录
强调	<code>\textbf {内容}</code>	语义化粗体强调
行内公式	<code>\$a^2+b^2=c^2\$</code>	嵌入段落中的短公式
独立公式	<code>\[... \]</code>	单独居中的无编号公式
引用	<code>\label</code> 、 <code>\ref</code>	自动维护编号
图片	<code>\includegraphics</code>	常配合 figure 环境
表格	<code>tabular</code> 、 <code>tabularx</code>	控制列、对齐和宽度

- 使用 `geometry` 设置 A4 页面边距；
- 使用 `fancyhdr` 生成统一页眉页脚；
- 使用 `tabularx`、`booktabs` 制作自适应宽度表格；
- 使用 `listings` 展示 Linux、Markdown、Java、JavaScript、Python 与 C++ 代码；
- 使用 `tcolorbox` 制作摘要框和提示框；
- 使用 `hyperref` 生成可点击目录与 PDF 元数据；
- 使用 TikZ 绘制阶段规划图，体现代码化制图能力。

这些功能说明，L^AT_EX 不仅可以输入数学公式，还能够完成结构化、可复用且风格统一的正式文档。

五、 个人编程语言基础与复习总结

5.1 已有语言基础概况

在此前的学习和项目开发中，我已经接触并使用过多种编程语言。其中 Java 和 JavaScript 是我较为熟练、实际使用较多的语言；SQL、HTML 与 CSS 是进行后端数据处理和 Web 开发时经常使用的配套技术。Python 与 C++ 已有一定基础，本次课程后将继续围绕科研计算和程序设计进行系统复习。

表 6: 个人编程语言与相关技术基础

语言或技术	掌握情况	主要用途	相关工具与框架
Java	熟练	后端业务开发、接口设计、数据处理	Spring Boot、MyBatis-Plus、Redis、OkHttp
JavaScript	熟练	Web 页面交互、前端工程化与浏览器 API	Vue 3、Vite、jQuery、Ant Design Vue
SQL	熟练使用	数据查询、统计分析与业务数据维护	MySQL、复杂查询、分组与关联统计
HTML/CSS	熟练使用	页面结构、布局、响应式界面	HTML5、CSS3、Less、JSP
Python	有基础	自动化、数据分析与科研原型	NumPy、pandas、Matplotlib
C++	有基础	算法练习与高性能程序	STL、CMake、编译与调试工具

5.2 Java 开发经验总结

Java是我实际项目中使用较多的语言。我能够使用面向对象思想组织业务代码，理解封装、继承、多态、集合、异常、泛型、I/O 和并发等常用机制。在后端开发中，我主要使用 Spring Boot 构建服务，结合 MyBatis-Plus 与 MySQL 完成数据访问，使用 Redis 处理缓存或队列任务，并通过 OkHttp、Gson 等组件调用和解析外部接口。

通过项目实践，我认识到 Java 开发不仅是实现功能，还需要关注分层设计、异常处理、日志记录、事务边界、参数校验和代码可维护性。例如，控制器负责接收请求，业务逻辑放在服务层，数据访问集中在持久层；对外部接口调用则要设置合理的超时并处理失败情况。这样的结构有利于测试、排错和后续迭代。

demo 9: Java 集合与 Stream API 示例

```
1 List<Integer> scores = List.of(86, 92, 78, 95);
2 double average = scores.stream()
3     .filter(score -> score >= 80)
4     .mapToInt(Integer::intValue)
5     .average()
6     .orElse(0.0);
7
8 System.out.printf("合格数据的平均值: %.2f\n", average);
```

5.3 JavaScript 与 Web 开发经验总结

JavaScript 主要用于浏览器端交互和前端工程化开发。我能够处理 DOM、事件、异步请求、JSON 数据和浏览器存储，理解作用域、闭包、原型、模块化、Promise 以及 `async/await` 等概念。在传统项目中可以结合 JSP 与 jQuery 完成页面开发，在现代项目中则可以使用 Vue 3、Vite、Ant Design Vue 和 Less 构建组件化界面。我本人的大部分实际经验也是基于 Vue 框架并结合 ant-design 或 element-ui 进行的项目。

实际开发中，前端代码需要同时考虑用户交互、接口状态和异常情况。例如，请求开始时显示加载状态，成功后更新页面，失败时给出明确提示，并在 `finally` 中恢复按钮状态。相比只追求“页面能运行”，模块划分、数据流向和错误处理更能体现工程质量。

demo 10: 原生 js 的异步请求示例

```
1  async function loadCandidate(id) {
2    try {
3      const response = await fetch(`/api/candidates/${id}`);
4      if (!response.ok) throw new Error("请求失败");
5      return await response.json();
6    } catch (error) {
7      console.error("加载数据时发生错误: ", error);
8      throw error;
9    }
10 }
```

5.4 Python 与 C++ 的定位

Python 语法简洁、生态丰富，适合数据处理、自动化、人工智能、科研原型和可视化；C++ 更接近底层，能够精细控制内存与性能，适合系统软件、高性能计算、图形开发和对实时性要求较高的场景。二者并非互相替代：科研中可以用 Python 快速验证思路，在性能瓶颈处使用 C++ 加速。

5.5 Python 知识框架

复习 Python 时，我将知识分为五层：

1. **语法基础**：变量、数值、字符串、列表、字典、集合、条件和循环；
2. **程序组织**：函数、模块、包、作用域、异常处理和文件读写；
3. **抽象能力**：类、对象、继承、迭代器、生成器和装饰器；

表 7: Python 与 C++ 对比

维度	Python	C++
类型系统	动态类型，运行时检查较多	静态类型，编译期检查较多
开发效率	语法简洁，原型开发快	语法复杂，需要管理更多细节
执行性能	通常较慢，可借助科学计算库	原生编译，性能与控制力较强
内存管理	自动垃圾回收为主	RAII 与智能指针，可精细控制
常见用途	数据分析、AI、脚本、Web	系统、高性能、游戏、嵌入式
科研策略	组织实验与处理数据	加速核心算法或对接底层设备

4. 工程实践：虚拟环境、依赖文件、日志、单元测试、类型标注和代码风格；
5. 科研生态：NumPy、pandas、Matplotlib、SciPy以及具体方向所需框架。

下面的程序演示“读取数据—统计—输出”的基本思路：

demo 11: Python 数据统计示例

```
1 from statistics import mean
2
3 scores = {"A": 86, "B": 92, "C": 78, "D": 95}
4 passed = {name: score for name, score in scores.items()
5           if score >= 80}
6
7 print(f"平均分: {mean(scores.values()):.2f}")
8 for name, score in sorted(passed.items(),
9                           key=lambda item: item[1], reverse=True):
10     print(name, score)
```

科研项目中应使用虚拟环境隔离依赖，例如执行 `python -m venv .venv`，激活后再安装所需包，并用 `requirements.txt` 或更现代的项目配置记录版本。原始数据、处理脚本和生成结果应分开保存，保证实验可追溯。

5.6 C++ 知识框架

C++复习的重点包括基本类型与控制结构、函数、引用与指针、类与对象、模板、标准模板库、异常、文件操作、资源管理和并发。现代 C++ 更强调 RAII、标准容器、算法和智能指针，应尽量避免手工管理裸指针带来的资源泄漏。

demo 12: C++ 标准库算法示例

```
1 #include <algorithm>
2 #include <iostream>
```

```

3 #include <numeric>
4 #include <vector>
5 #include <stdio.h>
6
7 int main() {
8     vector<double> values{3.2, 1.5, 4.8, 2.0};
9     sort(values.begin(), values.end());
10    const double sum = accumulate(values.begin(),
11                                  values.end(), 0.0);
12    cout << "mean = " << sum / values.size() << '\n';
13    return 0;
14 }

```

编译时应开启警告，并明确选择语言标准：

demo 13: C++ 编译示例

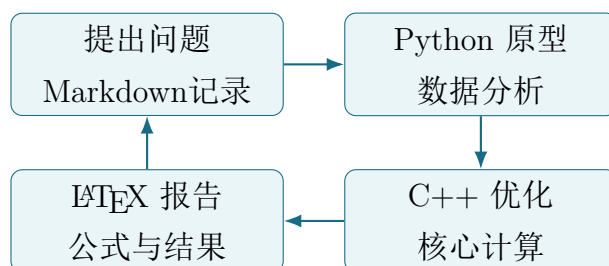
```

1 g++ -std=c++17 -Wall -Wextra -O2 main.cpp -o main
2 ./main

```

其中 `-Wall -Wextra` 有助于尽早发现可疑代码，`-O2` 开启常用优化。开发过程中可使用 CMake 管理多文件工程，并使用调试器和测试框架定位问题。实际使用中可使用类似 Clion 等 IDE 或者 VsCode+ 插件的方式来进行更方便的开发。

5.7 面向科研的联合 workflow



这一流程中，Linux 提供统一运行环境，Git 保存代码版本，Markdown 保存实验日志，Python 完成快速分析，C++ 处理性能敏感模块，最终由 L^AT_EX 生成正式论文或报告。工具之间形成闭环，才能真正提高科研效率。

六、 总结与后续计划

通过第一次课程，我认识到研究生阶段的关键不是被动完成安排，而是主动建立问题意识、时间规划和可持续的工作方法。课堂中关于科研流程、团队沟通和计算机管理的内容，为我之后的学习生活提供了明确提醒。

本次作业进一步把 Linux、Markdown、 $\text{\LaTeX}$ 与多种编程语言串联起来。Java 和 JavaScript 是我已有项目经验的重要基础，可分别承担后端服务与前端交互开发；SQL、HTML 与 CSS 为完整的 Web 开发提供支撑；Python 与 C++ 则分别兼顾科研原型的开发效率和核心计算的执行性能。掌握这些工具的目标并不是追求软件数量，而是根据任务选择合适技术，减少重复劳动、保证实验可复现，并把更多精力放在真正的研究问题上。

下一阶段，我计划按以下节奏持续学习：

1. 每周练习一组 Linux 命令，并在虚拟机或服务器中实际操作；
2. 使用 Markdown 记录课程笔记、论文阅读与实验日志；
3. 使用 $\text{\LaTeX}$ 完成后续课程报告，逐步学习参考文献、交叉引用和论文模板；
4. 巩固 Java 与 JavaScript 的工程化能力，同时用 Python 完成数据处理小项目，并复习 C++ 标准库与现代资源管理；
5. 为所有代码建立 Git 仓库，形成规范的版本记录和 README。

一句话总结

把研究任务拆解为可执行计划，把操作过程沉淀为可复用记录，把最终成果组织成可验证、可复现、可交流的文档。

参考资料

- [1] CommonMark. *CommonMark Specification*. <https://spec.commonmark.org/>
- [2] GitHub. *GitHub Flavored Markdown Specification*. <https://github.github.com/gfm/>
- [3] The Linux man-pages project. *Linux manual pages*. <https://www.kernel.org/doc/man-pages/>
- [4] The L^AT_EX Project. *LaTeX documentation*. <https://www.latex-project.org/help/documentation/>
- [5] Python Software Foundation. *Python Documentation*. <https://docs.python.org/3/>
- [6] ISO C++ Foundation. *Standard C++ resources*. <https://isocpp.org/>
- [7] 课程讲义：学业基本情况介绍与计算机基本使用，2026 年 7 月 14 日。